

UNIVERSITY OF BUEA

FACULTY OF
ENGINEERING AND
TECHNOLOGY
(FET)

PO BOX 63



REPUBLIC OF CAMEROON

PEACE-WORK-
FATHERLAND

BUEA, SOUTH WEST REGION

TASK 1 REPORT

MOBILE APP DEVELOPMENT PROCESS

COURSE TITLE: *INTERNET PROGRAMMING AND
MOBILE PROGRAMMING*

COURSE CODE: *CEF 440*

COURSE INSTRUCTOR: *DR NKAMENI VALERY*

➤ GROUP 13 MEMBERS

1. IMELE AZAFA JOSE DE KAJOS -----FE23A068
2. AYUK AMELIA AGBOR OROCK----- FE23A020
3. NGOME FERNANDEZ MANYAKA----- FE23A111
4. TADJEU NGNINTEDEM AUBRY-JUNEX----- FE23A148
5. BESSONG TAKANG TABOT WILLY----- FE23A025

*A Comprehensive Review of Mobile App Types, Languages, Frameworks,
Architectures, Requirements & Cost Estimation*

DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC YEAR 2025 - 2026

1 | CEF440

Table of Contents

Table of Contents	2
Introduction	5
1. Types of Mobile Applications	6
1.1 Native Apps	6
1.2 Progressive Web Apps (PWA)	6
1.3 Hybrid Apps	7
1.4 Comparison Summary	8
2. Mobile App Programming Languages	9
2.1 Swift (iOS)	9
2.2 Kotlin (Android)	9
2.3 Java (Android).....	9
2.4 Dart (Flutter).....	10
2.5 JavaScript and TypeScript (Cross-platform).....	10
2.6 C# (Xamarin / .NET MAUI)	10
2.7 Language Comparison Table.....	10
3. Mobile App Development Frameworks	12
3.1 React Native	12
3.2 Flutter	13
3.3 Ionic	14
3.4 Xamarin / .NET MAUI.....	15
3.5 Framework Comparison Table.....	16
3.6 Recommendations by Use Case.....	17
4. Mobile Application Architectures and Design Patterns	18
4.1 Architectural Patterns.....	18
4.1.1 Model-View-Controller (MVC)	18
4.1.2 Model-View-Presenter (MVP)	19
4.1.3 Model-View-ViewModel (MVVM).....	20
4.1.4 Clean Architecture.....	21
4.1.5 BLoC Pattern (Flutter)	21
4.2 Architecture Comparison Table	22
4.3 Design Patterns.....	22

4.3.1 Creational Patterns.....	22
4.3.2 Structural Patterns	22
4.3.3 Behavioural Patterns	23
4.4 Design Patterns Summary Table.....	23
5. Collecting and Analysing User Requirements (Requirements Engineering)	24
5.1 Types of Requirements.....	24
5.2 The Requirements Gathering Process	25
Step 1 – Define the App's Idea and Purpose	25
Step 2 – Identify and Engage Stakeholders.....	25
Step 3 – Run Market and Competitor Analysis	25
Step 4 – Create User Personas and Scenarios	25
Step 5 – Gather Functional and Non-Functional Requirements	25
5.3 Requirements Elicitation Techniques.....	26
Interviews.....	26
Surveys and Questionnaires	26
Contextual Inquiry / Observation.....	26
Focus Groups	26
Workshops	26
User Stories	26
Prototyping and Wireframing	26
5.4 Requirements Analysis Techniques.....	27
5.5 Requirements Documentation	27
5.6 Requirements Validation.....	28
CASE STUDY : How to Design a Use Case for App Requirements Documents.....	28
Write an App Requirements Document	29
6. Estimating Mobile App Development Cost	32
6.1 Key Cost Factors.....	32
App Complexity and Number of Features.....	32
Platform Choice.....	32
UI/UX Design	32
Backend Development and API Integrations.....	32
Team Location and Composition	32
Maintenance and Post-Launch Costs	33
6.2 Cost Estimation Techniques	33

App Development Cost Formula.....	33
Story Point Estimation (Agile)	33
Three-Point Estimation (PERT)	33
Function Point Analysis (FPA)	34
Analogous Estimation	34
6.3 Step-by-Step Cost Estimation Process	34
6.4 Typical Cost Ranges.....	34
6.5 Cost Distribution Across Development Phases.....	35
6.6 Tactics for Reducing Development Costs.....	35
7. Conclusion	36
References	37

Introduction

Mobile application development is the process of creating software applications designed to run on mobile devices such as smartphones and tablets. These applications serve a wide variety of purposes from entertainment and social networking to productivity, e-commerce, and enterprise solutions. As mobile technology continues to evolve rapidly, developers and organisations must stay informed about the best tools, languages, frameworks, and methodologies available.

This report, provides a comprehensive study of all six task areas: the major types of mobile apps and their trade-offs, programming languages for mobile development, frameworks for cross-platform development, architectural patterns, requirements engineering, and cost estimation techniques.

1. Types of Mobile Applications

Mobile applications are broadly classified into three major types based on how they are built, deployed, and interact with the underlying operating system and hardware. Each type presents distinct trade-offs between performance, cost, accessibility, and development effort.

There are three kinds of mobile applications. They are;

- Native apps.
- Web apps.
- Hybrid apps.

1.1 Native Apps

Native apps are developed specifically for a single mobile platform — either iOS or Android — using the platform's official programming languages and development tools. Because they are purpose-built for the target OS, they offer the best possible performance, user experience, and full access to all hardware capabilities.

- Languages: Swift or Objective-C for iOS; Kotlin or Java for Android
- Direct access to device hardware: camera, GPS, accelerometer, Bluetooth, NFC
- Best performance and user experience of all app types
- Must be submitted to platform stores (App Store, Google Play) for distribution
- Requires a separate codebase and development team for each platform
- Examples: Spotify, WhatsApp, Snapchat, Instagram, Google Maps (Android)

1.1.1. Advantages of Native Apps

- Access to device-specific features
- No internet connection needed
- Impressive users experience on Android or IOS

1.1.2. Disadvantages of Native Apps

- Multiple code bases
- Higher budget costs
- Longer build times
- Maintenance can be complicated
- Native apps tend to be less discoverable than hybrid or web apps

1.2 Progressive Web Apps (PWA)

Progressive Web Apps (PWAs) are web applications built using modern web technologies that deliver an app-like experience directly through a browser, without requiring installation from an app store. They leverage Service Workers for offline capability and can be added to a device's home screen.

- Built with standard web technologies: HTML5, CSS3, JavaScript
- Accessible through any browser on any device or operating system
- Offline or low-network support via Service Workers
- No app-store submission required; can be added to home screen directly
- Limited access to native device features (no Bluetooth, NFC, etc.)
- Examples: Twitter Lite, Starbucks PWA, Pinterest PWA, Trivago

1.2.1 Advantages of Web Apps

- Easy discoverability
- Faster development time than Native
- Simple to Maintain
- Lower development costs
- They work on all Platforms
- Updates made to the application are immediately available

1.2.2 Disadvantages of Web Apps

- Slower performance than native
- Unable to work offline
- Cannot access platform features
- Unable to access UI and match style like native apps

1.3 Hybrid Apps

Hybrid apps combine elements of both native and web applications. They are essentially web apps packaged inside a native container (using frameworks such as Cordova or Capacitor), allowing them to be distributed through app stores while being built with web technologies. This approach enables a single codebase to target multiple platforms.

- Built with web technologies (HTML, CSS, JavaScript) wrapped in a native shell
- Single codebase deployed to iOS, Android, and sometimes Windows Phone
- Access to native device features via bridge plugins (Cordova, Capacitor)
- Performance sits between native and pure web apps
- Examples: Facebook (early), Instagram (early), Uber (certain components), LinkedIn, Gmail

1.3.1 Advantages of Hybrid Apps

- Less expensive to sell in app store
- simple to maintain
- Lower development costs
- Targets multiple platforms

- Allows apps to match look of native application and take advantage of the platform's UI style

1.3.2 Disadvantages of Hybrid Apps

- Slower performance than native
- Can't access platform features due to WebView restrictions

1.4 Comparison Summary

Feature	Native	PWA	Hybrid
Performance	Excellent – optimized for platform	Good – runs in browser	Moderate – depends on WebView
UX/UI Quality	Best – platform native components	Good – responsive design	Good – mimics native look
Device Access	Full hardware access	Limited (no BT, NFC etc.)	Moderate via plugins
App Store	Required	Not required	Required
Offline Support	Yes	Yes (Service Workers)	Yes
Development Cost	High – separate codebase per platform	Low – single web codebase	Moderate – single codebase
Time to Market	Slow	Fast	Moderate
Codebase	Per platform	Single	Single

Recommendation: Choose native for performance-critical or hardware-intensive apps; PWA for broad reach with minimal cost; hybrid for moderate-performance apps needing multi-platform coverage quickly.

2. Mobile App Programming Languages

The programming language chosen for a mobile project significantly affects development speed, runtime performance, available talent, long-term maintainability, and the ecosystem of libraries. Below is a review of the most widely used languages in mobile development.

2.1 Swift (iOS)

Swift is a modern, type-safe, compiled language developed by Apple and released in 2014. It replaces Objective-C as the primary language for iOS, macOS, watchOS, and tvOS development. Its clean syntax, memory safety features, and high performance make it the standard choice for any iOS-targeted project.

- Compiled language – performance close to C++
- Strong typing and Automatic Reference Counting (ARC) reduce bugs and memory leaks
- Fully interoperable with legacy Objective-C codebases
- Supports closures, generics, and modern programming paradigms

2.2 Kotlin (Android)

Kotlin became Google's officially preferred Android language in 2017. It is concise, safe, and fully interoperable with Java, meaning all existing Java libraries and frameworks can be used without modification. Its null-safety type system and coroutine support have made it the dominant language for new Android projects.

- 100% interoperable with Java – reuse all Java libraries
- Null safety built into the type system prevents NullPointerExceptions
- Supports coroutines for clean, readable asynchronous code
- Concise – reduces boilerplate code significantly compared to Java

2.3 Java (Android)

Java was the original language of Android and still powers a huge portion of existing Android applications. While Kotlin is now preferred for new projects, Java's massive ecosystem, deep community support, and extensive documentation make it relevant for maintaining legacy codebases and for developers who are already fluent in Java.

- Large existing codebase and mature ecosystem of libraries
- Verbose but well-understood and widely documented
- Runs on the JVM – optimised for Android

2.4 Dart (Flutter)

Dart is an open-source language developed by Google, used almost exclusively with the Flutter framework. It compiles to native ARM code, enabling excellent runtime performance. Its reactive programming model and hot-reload feature significantly accelerate UI development.

- AOT (Ahead-of-Time) compilation to native ARM code – excellent performance
- Hot reload accelerates UI iteration without losing app state
- Strongly typed and object-oriented
- Growing community, though smaller than JavaScript or Java

2.5 JavaScript and TypeScript (Cross-platform)

JavaScript is the most widely used programming language in the world and powers cross-platform mobile frameworks like React Native and Ionic. TypeScript, a typed superset of JavaScript maintained by Microsoft, improves code quality and developer experience while remaining fully compatible with the JavaScript ecosystem.

- Largest developer community globally
- TypeScript adds static typing, reducing runtime errors
- JIT (Just-in-Time) compiled – slightly lower performance than native languages
- Versatile: same language used for web, mobile (React Native, Ionic), and back-end (Node.js)

2.6 C# (Xamarin / .NET MAUI)

C# is a multi-paradigm language by Microsoft, primarily used in mobile development via the Xamarin framework (now evolving into .NET MAUI). It is the best choice for teams already embedded in the Microsoft / Azure ecosystem, offering strong tooling support through Visual Studio.

- Object-oriented with strong type system
- AOT compilation via Mono runtime provides good performance
- Extensive libraries and deep Azure / .NET integration
- Best suited for enterprise environments using Microsoft technology stacks

2.7 Language Comparison Table

Language	Platform	Performance	Learning Curve	Dev Speed	Community
Swift	iOS / Apple	Excellent	Moderate	Quick with rich libs	Large
Kotlin	Android	Excellent	Moderate	Fast, concise syntax	Large
Java	Android	Good	Steep	Moderate	Very Large
Dart	Cross-platform	Very Good (AOT)	Low–Moderate	Fast (hot reload)	Growing
JavaScript	Cross-platform	Moderate	Low	Varies by framework	Massive
TypeScript	Cross-platform	Moderate	Low–Moderate	Improved via typing	Very Large
C#	Cross-platform	Good (AOT)	Moderate	Quick in VS	Medium–Enterprise

Table: Comparison analysis of major mobile programming language

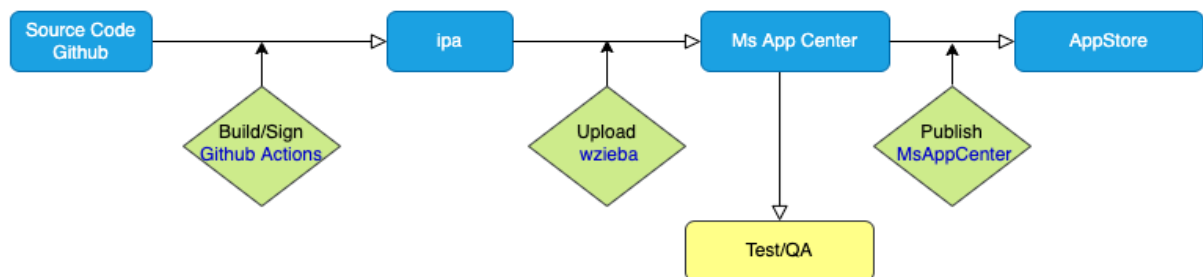
3. Mobile App Development Frameworks

A mobile development framework provides the tools, libraries, abstractions, and runtime environment that simplify building cross-platform applications. This section compares the four most prominent frameworks — React Native, Flutter, Ionic, and Xamarin/.NET MAUI — across the key dimensions of language, performance, cost and time to market, UX and UI, complexity, and community support.

3.1 React Native

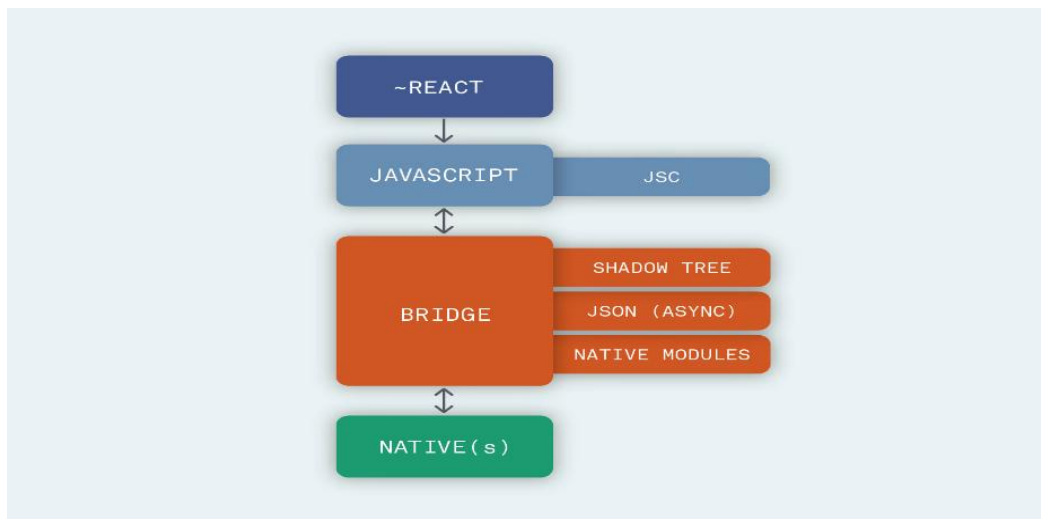
Created by Meta (Facebook) in 2015 and open-sourced immediately, React Native is one of the most widely adopted cross-platform mobile frameworks. It uses JavaScript (or TypeScript) and allows developers with web backgrounds to build iOS and Android applications using a single codebase that maps to actual native UI components.

Diagram: React Native Workflow



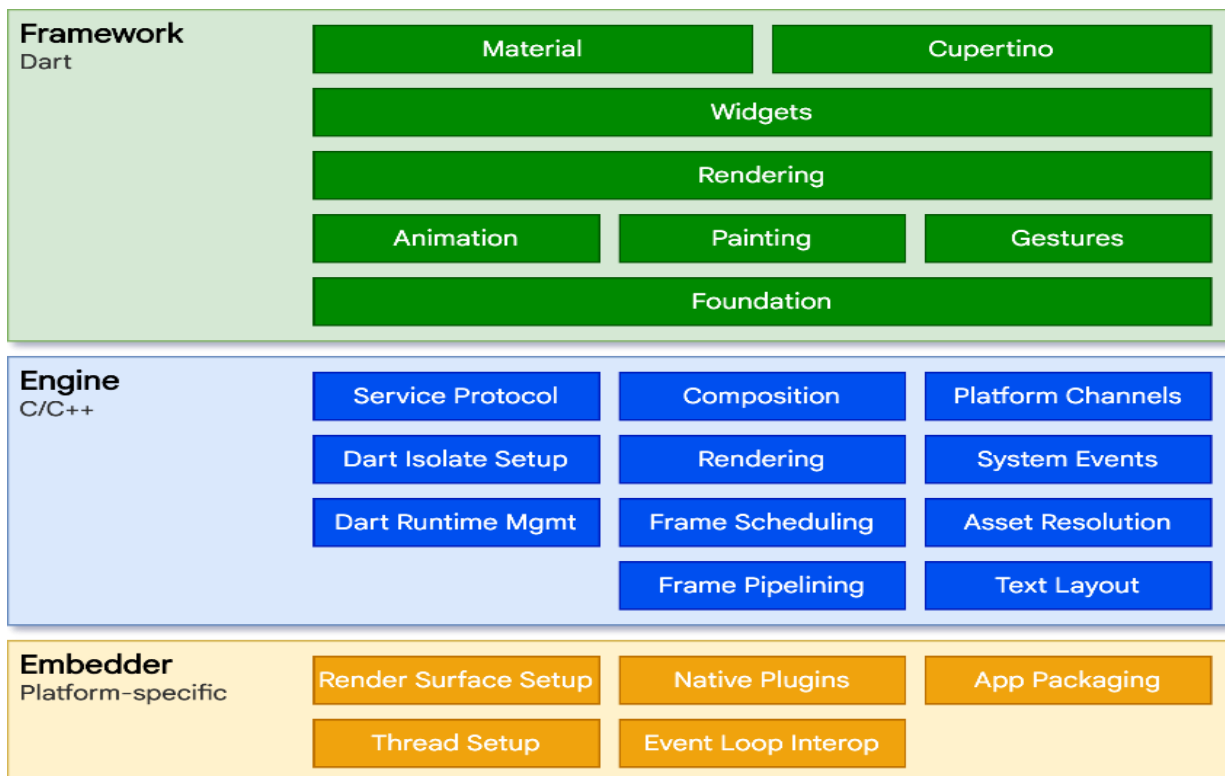
- **Language:** JavaScript / TypeScript
- **Architecture:** Bridge-based – JavaScript thread communicates with native components via an asynchronous bridge. The newer JSI (JavaScript Interface) architecture reduces bridge overhead.
- **Performance:** Near-native for most use cases. Achieves 55–60 FPS. Complex animations may drop due to bridge latency. Memory usage ~20–30% higher than pure native.
- **Cost & Time to Market:** Reduces development cost by 30–50% vs. separate native apps. Hot reload speeds iteration. A medium-complexity app typically costs \$50K–\$120K and takes 4–8 months.
- **UX & UI:** Uses actual native UI components (View, Text, Image), giving a truly native look and feel. Platform-specific styling tweaks may still be needed.
- **Complexity:** Moderate. Familiar to web developers, but advanced features require native module integration. The bridge can create bottlenecks in heavy animation scenarios.

- **Community:** Very large. 110K+ GitHub stars, 500K+ Stack Overflow questions. Backed by Meta with regular updates and a massive open-source ecosystem.
- **Used by:** Facebook, Instagram, Microsoft Teams, Shopify, Coinbase, Uber Eats



3.2 Flutter

Flutter was released by Google in 2017 as an open-source UI toolkit. It compiles Dart code to native ARM code and uses its own high-performance rendering engine (Skia, now Impeller), bypassing native UI components entirely. This gives Flutter pixel-perfect consistency across all platforms from a single codebase.



- **Language:** Dart
- **Architecture:** Compiled to native ARM code; own rendering engine (Skia/Impeller). No JavaScript bridge. UI is drawn directly by the engine rather than delegated to OS widgets.
- **Performance:** Excellent. Consistently achieves 60 FPS; supports 120 FPS on high-end devices. Faster startup than React Native. More memory-efficient due to direct native compilation.
- **Cost & Time to Market:** Single codebase for iOS, Android, Web, and Desktop. A medium-complexity app costs \$50K–\$110K and takes 4–7 months. Maintenance is ~10–15% of initial cost per year.
- **UX & UI:** Highly customisable pixel-perfect widgets. Uses Material Design (Android-style) and Cupertino (iOS-style) widget sets. No native components, so all UI is drawn by Flutter.
- **Complexity:** Low to moderate. Dart is beginner-friendly. The widget-tree paradigm and reactive model require a learning curve, but tooling is excellent.
- **Community:** Fast growing. 160K+ GitHub stars. Strong package ecosystem at pub.dev. Backed by Google with active developer meetups and conferences.
- **Used by:** Google Pay, BMW App, Alibaba, Nubank, eBay Motors

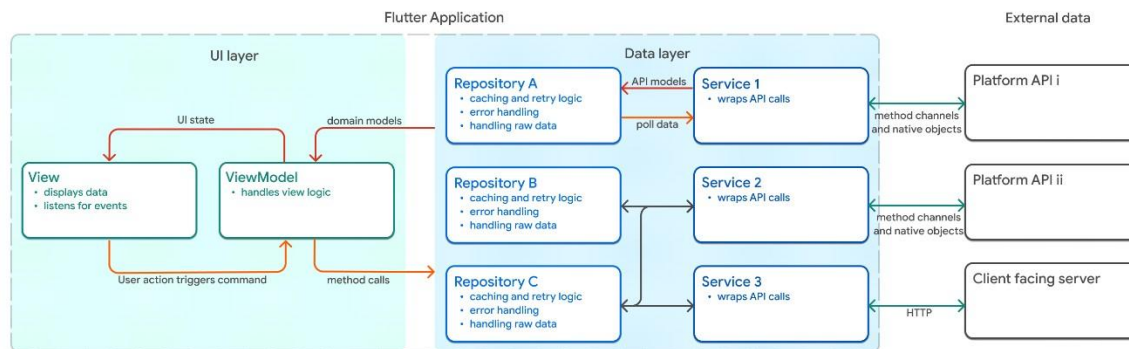
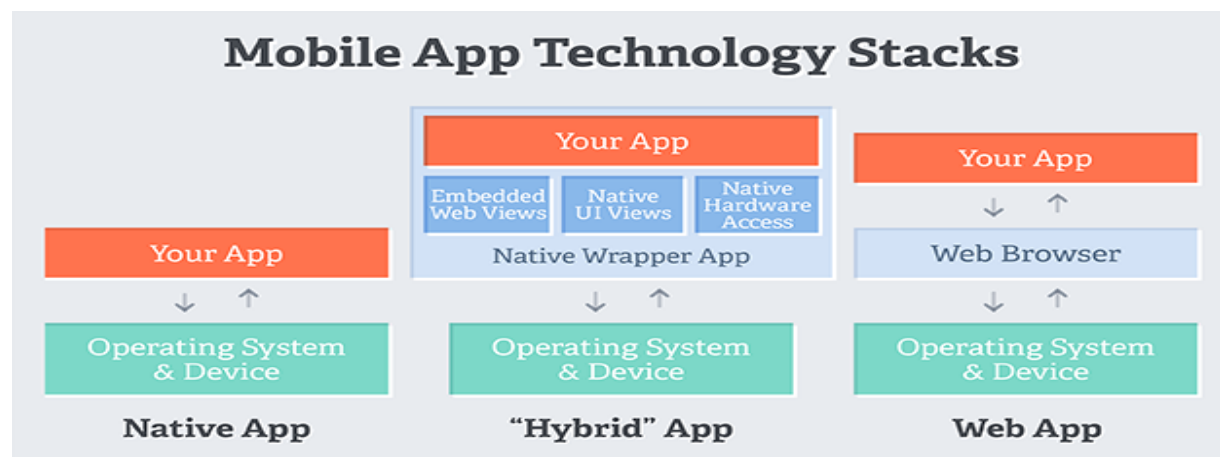


Figure: Flutter Architecture

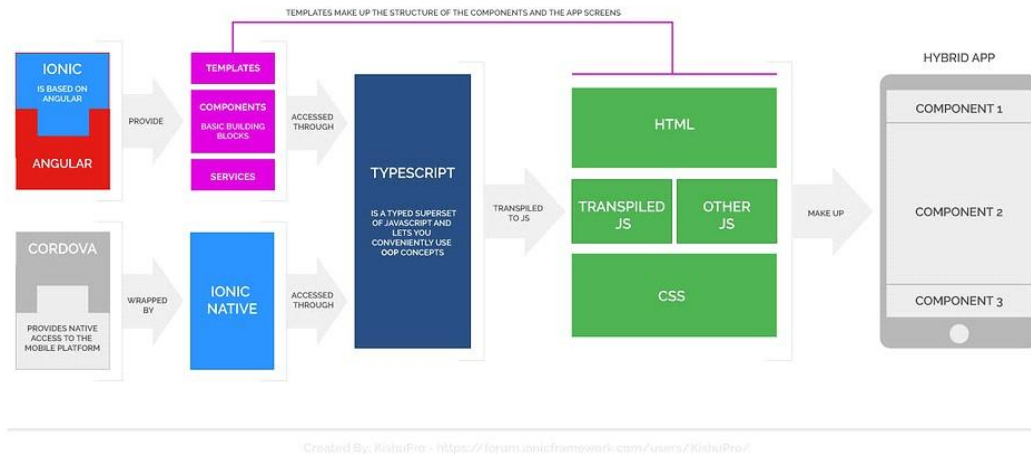
3.3 Ionic

Ionic, originally developed by Drifty Co. in 2013, is an open-source framework for building hybrid and Progressive Web Apps using standard web technologies — HTML, CSS, and JavaScript — with support for Angular, React, or Vue as the UI framework. Apps run inside a WebView on mobile devices.



- **Language:** HTML, CSS, JavaScript / TypeScript (with Angular, React, or Vue)
- **Architecture:** WebView-based – the app runs in an embedded browser window; Capacitor (successor to Cordova) provides native device access.

A BIRD'S EYE VIEW OF IONIC AND RELATED TECHNOLOGIES



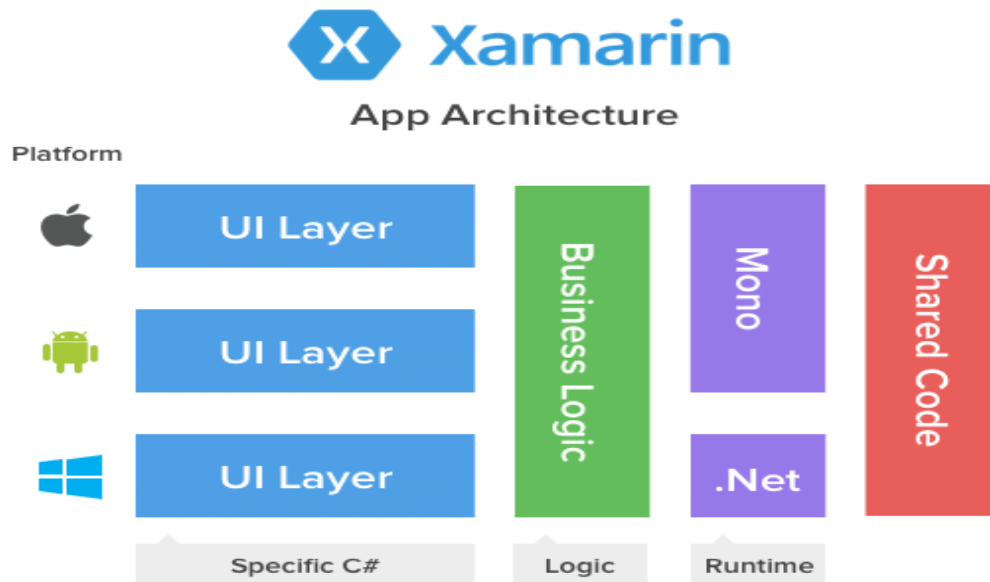
- **Performance:** Moderate. Startup is slower than Flutter/RN; typically 30–60 FPS. Complex interactions can lag due to WebView rendering limitations. Uses the most memory of the three.
- **Cost & Time to Market:** Cheapest and fastest option. Any web developer can start immediately. A medium-complexity app costs \$40K–\$90K and takes 3–6 months. Maintenance ~5–10%/year.
- **UX & UI:** Web-based components styled to mimic native appearance. Less fluid animations. Suitable for content-centric apps and MVPs where native fidelity is not critical.
- **Complexity:** Low. Lowest barrier to entry, especially for web developers. Increases when integrating native features beyond Capacitor's built-in plugins.
- **Community:** Active but smaller. Solid documentation and community forums. GitHub: 49K+ stars.
- **Used by:** MarketWatch, Sworkit, and many enterprise MVPs and internal tools

3.4 Xamarin / .NET MAUI

Xamarin, acquired by Microsoft, allows C# developers to build cross-platform mobile applications using the .NET ecosystem. It has evolved into .NET MAUI (Multi-platform App UI), which extends cross-platform support to desktop as well. It is the preferred choice for enterprise teams already using Microsoft's technology stack.

- **Language:** C#
- **Architecture:** AOT (Ahead-of-Time) compilation via the Mono runtime; .NET MAUI provides a unified cross-platform UI layer.
- **Performance:** Good – near-native via AOT compilation, though the Mono runtime adds overhead compared to true native or Flutter.
- **Cost & Time to Market:** Moderate. Free framework, but enterprise tooling (Visual Studio) may add cost. Best for teams with existing C# expertise.

- **UX & UI:** Can use native controls (Xamarin.Native) or a shared UI (MAUI). Native controls offer better platform fidelity.
- **Complexity:** Moderate to high for teams unfamiliar with .NET. Deep integration with Azure and Microsoft services.
- **Community:** Medium-sized but enterprise-focused. GitHub: ~8K stars. Microsoft backed with strong enterprise documentation.
- **Used by:** Enterprise applications in the Microsoft ecosystem



3.5 Framework Comparison Table

Framework	Language	Performance	Cost / Time	UX & UI	Complexity	Community
React Native	JS / TS	Near-native (bridge)	Low / Fast	True native components	Moderate	Very Large – Meta backed
Flutter	Dart	Excellent (ARM, Skia)	Low / Fast	Custom pixel-perfect widgets	Low–Moderate	Large & Growing – Google

Framework	Language	Performance	Cost / Time	UX & UI	Complexity	Community
Ionic	JS / TS (HTML/CSS)	Moderate (WebView)	Very Low / Fastest	Web components, less native feel	Low	Moderate – stable
Xamarin / .NET MAUI	C#	Good (AOT Mono)	Moderate	Native or shared UI	Moderate–High	Medium – Microsoft enterprise

3.6 Recommendations by Use Case

Use Case	Best Framework	Why?
MVP for Startups	React Native	Fast, JS ecosystem, native-like
Gaming/Media Apps	Flutter	60fps animations, custom widgets
Enterprise Integration	Xamarin	.NET compatibility, AOT compilation
Simple PWA	Ionic	Web skills reuse, low cost

- Choose React Native for: Projects with moderate complexity; teams with JavaScript/React experience; apps requiring native-like performance with a fast time to market.
- Choose Flutter for: High-performance apps with complex animations and custom UI; apps needing consistent pixel-perfect design across all platforms.
- Choose Ionic for: Hybrid apps and MVPs requiring rapid development; businesses wanting PWAs; developers comfortable with web technologies.
- Choose Xamarin / .NET MAUI for: Enterprise projects already using Microsoft's C#/.NET stack; applications requiring deep Azure integration.

4. Mobile Application Architectures and Design Patterns

Architecture defines how an application is structured internally — how data flows, how the UI updates, and how different components communicate. Choosing the right architecture is fundamental to ensuring an app is scalable, maintainable, and testable. Without deliberate architectural planning, applications grow increasingly difficult to debug, extend, and test — a common challenge for student projects.

I. Definition

Mobile app architecture is the blueprint for building an app, defining how its components (like UI, backend, and database) interact to **process inputs** and **produce outputs**, ensuring scalability, maintainability, and a smooth user experience.

Why is App Architecture Essential?

Quality architecture helps with risk management and enables cost reductions. An application with robust, well-planned architecture is more likely to succeed in its target market. Any mobile app project starts with the planning and designing phase, and choosing the right architecture is a core priority. An insufficient approach to this step can slow down the development process and make it more extensive. It can also lead to various performance issues and system failures.

II. Examples of Mobile App Architecture

Some Examples of mobile app architecture include; **Mobile View Controller (MVC)**, **Mobile View Presenter (MVP)**, **Mobile View View-Model (MVVM)**, **Layered Architecture** and **Clean Architecture**. Amongst these, the most commonly used architectures include **MVC**, **MVP**, **MVVM** which will be discussed in detail in this section.

Note: These patterns are widely used to moderate complex codes and simplify UI code by making it neater and more manageable. MV(X) architectures divide the visualizing, processing, and data management functions for UI applications, which increases an app's modularity, flexibility, and testability.

Let us take a closer look at each model to understand their differences.

4.1 Architectural Patterns

4.1.1 Model-View-Controller (MVC)

MVC is the foundational pattern widely used in early iOS development (UIKit) and simple Android apps. It separates the application into three components:

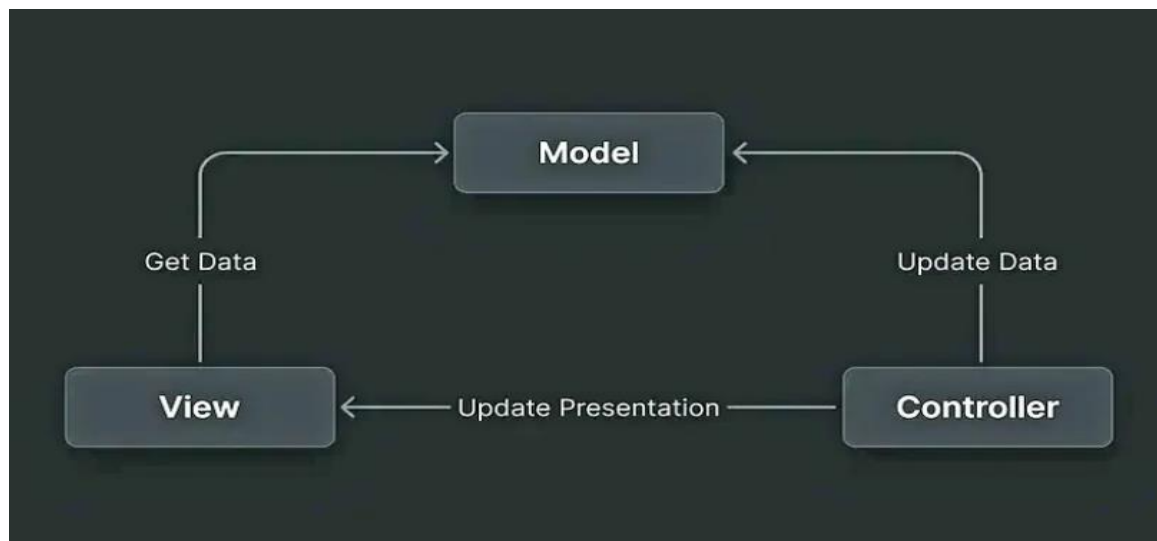
- Model: manages data, business logic, and state changes.
- View: handles all UI elements and presents data to the user.

- **Controller:** acts as an intermediary — processes user input, updates the model, and instructs the view to refresh.

Strengths: Simple to understand, supports rapid development, enables multiple views for the same model, allows parallel development.

Weaknesses: Controller tends to become bloated (the 'Massive View Controller' anti-pattern), limited testability since controller is tightly coupled to the view.

Best for: Small to medium iOS apps. Examples: Pinterest and Twitter (older versions), Instagram (early versions).



4.1.2 Model-View-Presenter (MVP)

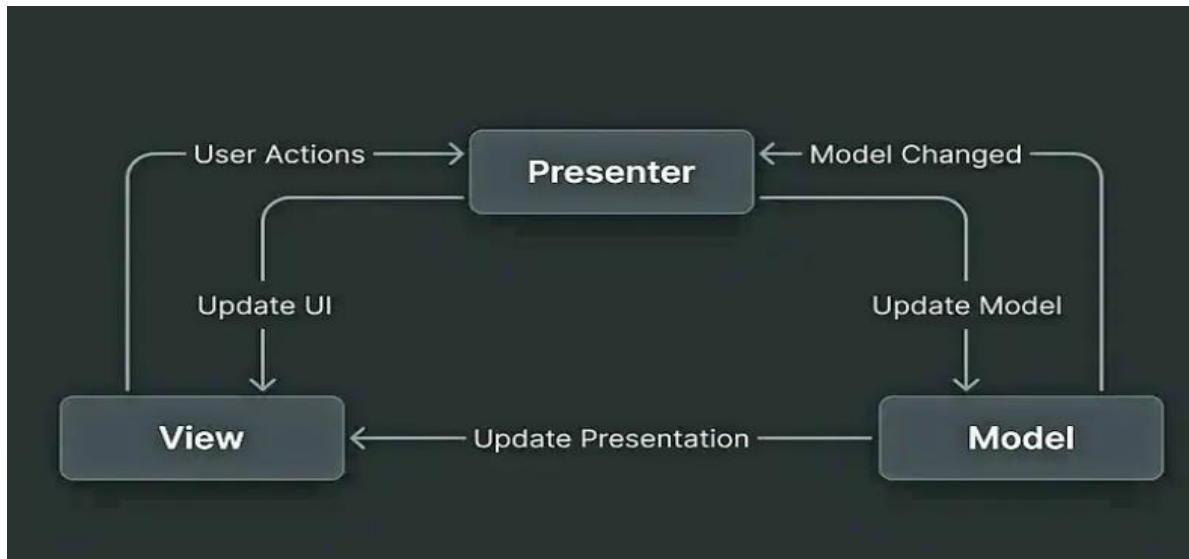
MVP is derived from MVC and replaces the Controller with a Presenter. The View is made passive — it only renders what the Presenter tells it to render. The Presenter handles all UI logic and communicates with the Model.

- Presenter manipulates the model and updates the view based on user actions.
- View communicates with Presenter via a well-defined interface (contract).
- Presenter has no Android/iOS framework dependency — easy to unit test.

Strengths: High testability, improved maintainability, clear separation of concerns, better handling of complex UI logic.

Weaknesses: Tight coupling between Presenter and View interface, more boilerplate than MVC, no lifecycle awareness.

Best for: Medium-complexity Android apps. Examples: Spotify (older Android), Uber (earlier versions), Google Apps (legacy Gmail/YouTube).



4.1.3 Model-View-ViewModel (MVVM)

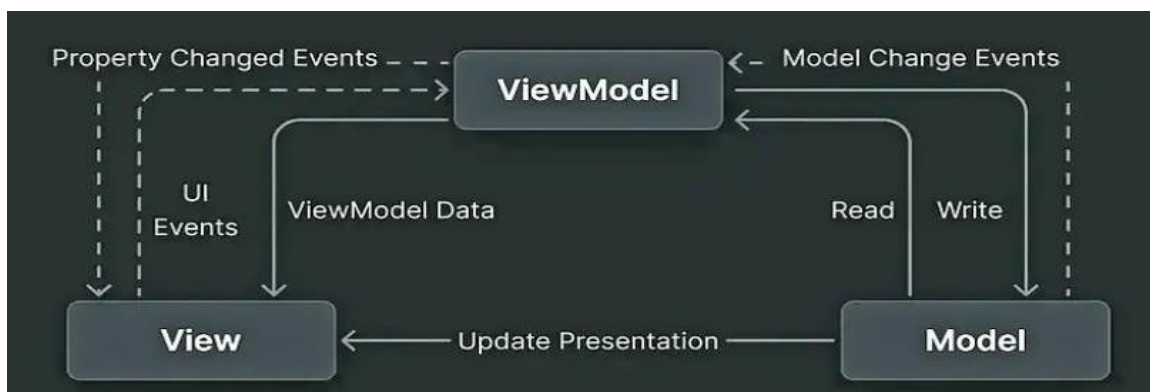
MVVM is the modern standard for Android (Jetpack ViewModel + LiveData/StateFlow) and iOS (SwiftUI + Combine). The ViewModel exposes observable data streams that the View binds to — when the data changes, the UI updates automatically via two-way data binding.

- ViewModel: exposes observable state; contains business and presentation logic but no direct reference to the View.
- View: binds to ViewModel observables and reacts to state changes.
- Model: handles data and business logic.

Strengths: Lifecycle-aware, highly testable (no UI dependency in ViewModel), scalable, easier parallel development, reactive programming support.

Weaknesses: Can introduce over-engineering for small apps, steeper learning curve, debugging data bindings can be challenging.

Best for: Medium to large apps with complex UI state. Examples: Gmail, YouTube, Google Maps (modern Android), Netflix, Airbnb, Microsoft Outlook.



4.1.4 Clean Architecture

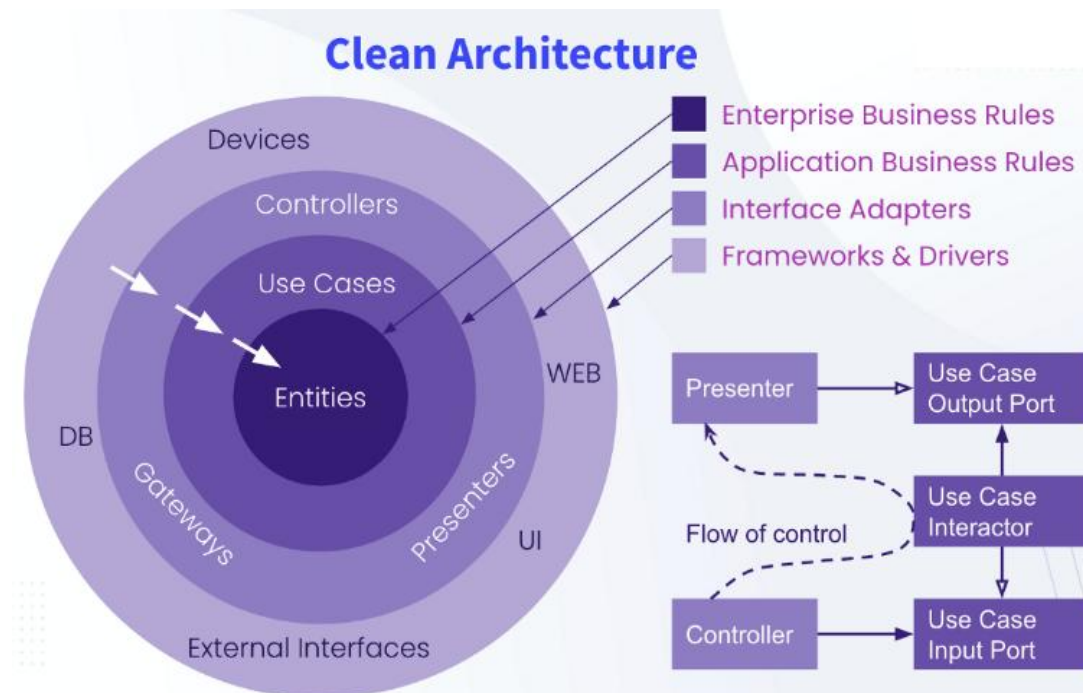
Clean Architecture (proposed by Robert C. Martin) separates the codebase into three independent layers that enforce a strict dependency rule: outer layers depend on inner layers, never the reverse.

- Presentation Layer: UI components, ViewModels, Activities/Fragments.
- Domain Layer (Use Cases): Business rules — independent of frameworks and databases.
- Data Layer: Repository implementations, remote API clients, local database adapters.

Strengths: Highly testable and maintainable, framework-independent domain logic, long-term scalability, supports multiple data sources cleanly.

Weaknesses: More upfront setup complexity, can be over-engineering for small apps.

Best for: Large enterprise applications. Combines well with MVVM at the Presentation layer.



4.1.5 BLoC Pattern (Flutter)

The Business Logic Component (BLoC) pattern was created specifically for Flutter apps. It completely separates business logic from the UI using Dart Streams: the UI sends Events to the BLoC; the BLoC emits States; the UI rebuilds when new states are emitted.

- Highly predictable and testable state management
- Clear contract: UI → Events → BLoC → States → UI
- Scales very well for large Flutter projects

Weaknesses: Verbose for simple use cases; requires understanding of reactive streams.

4.2 Architecture Comparison Table

Pattern	Separation	Testability	Complexity	Best Used With	Use Case
MVC	Moderate	Moderate	Low	iOS UIKit	Small, simple apps
MVP	High	High	Moderate	Android (legacy)	Medium Android apps
MVVM	High	High	Moderate	Android Jetpack / SwiftUI	Medium–large apps with complex UI
Clean Architecture	Very High	Excellent	High	Any platform	Large enterprise apps
BLoC	Very High	Excellent	Moderate–High	Flutter	Complex Flutter apps

4.3 Design Patterns

Beyond architectural patterns, individual design patterns solve recurring coding problems and promote code reusability, modularity, and maintainability. The following six patterns are the most relevant in mobile development:

4.3.1 Creational Patterns

- Singleton: Ensures a class has only one instance with a global access point. Used for shared services like network managers, logging, and database connections. Risk: tight coupling if overused.
- Factory Method: Centralises object creation logic, enabling flexible and extensible instantiation without modifying existing code. Used for creating different types of notifications, ride options (Uber), or payment methods.

4.3.2 Structural Patterns

- Adapter: Acts as a bridge between incompatible interfaces, enabling integration of legacy code or third-party libraries without modification. Extensively used in Android RecyclerView adapters.

- Composite: Manages tree structures of objects uniformly — useful for building nested UI component trees.

4.3.3 Behavioural Patterns

- Observer: Implements event-driven communication where multiple components react to state changes. Foundation of LiveData (Android), Combine (iOS), and any real-time notification system.
- Strategy: Defines a family of interchangeable algorithms and selects one at runtime. Used in payment processing systems (PayPal, Stripe) and search sorting.
- Repository: Provides a clean data access abstraction that decouples the data source (API, local DB, cache) from business logic. Best combined with MVVM or Clean Architecture.

4.4 Design Patterns Summary Table

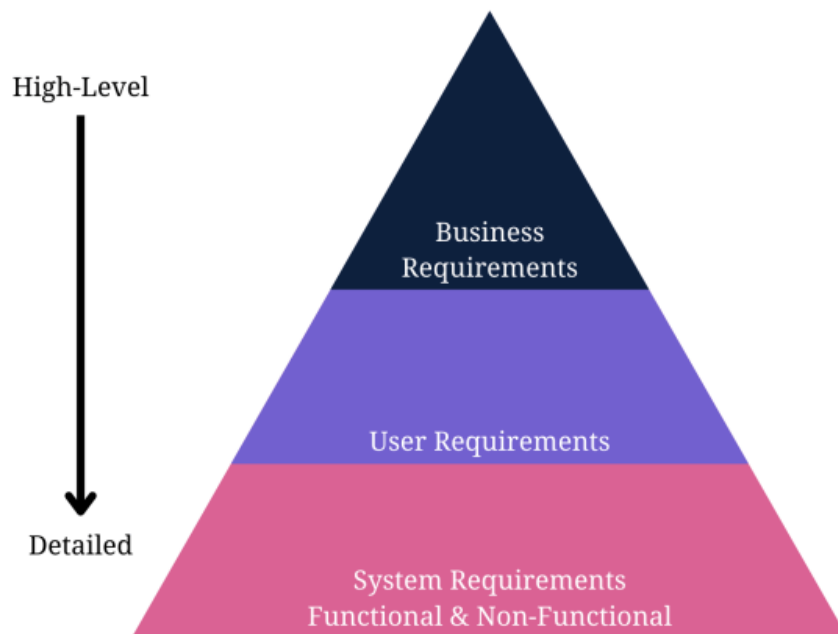
Pattern	Purpose	Key Benefit	Real-World Examples
Singleton	Single shared instance management	Memory efficiency, global access	Spotify (Playback Mgr), Google Maps (Location)
Factory Method	Flexible object creation	Loose coupling, easy extension	Uber (ride types), Amazon (shipping options)
Observer	Event-driven updates	Decouples emitters and listeners	Facebook (live notifications), WhatsApp (status)
Adapter	Bridge between incompatible interfaces	Reusability, legacy integration	Android RecyclerView, Netflix (movie list)
Strategy	Interchangeable algorithms at runtime	Modularity, flexible business logic	PayPal (payment methods), Google (sorting)
Repository	Clean data access abstraction	Maintainability, testable data layer	Netflix (cache + API), Google Drive (sync)

5. Collecting and Analysing User Requirements (Requirements Engineering)

Requirements Engineering (RE) is the systematic discipline of identifying, documenting, validating, and managing the requirements of a software system. For mobile apps, thorough RE is essential because changes discovered late in development are exponentially more expensive to address than those caught during the planning phase.

5.1 Types of Requirements

- **Functional Requirements:** Define what the app must do — specific features and behaviours (e.g., user login, product search, payment processing, push notifications).
- **Non-Functional Requirements:** Define how well the app performs — speed, security, scalability, reliability, battery efficiency, accessibility.
- **Business Requirements:** High-level goals that ensure the app aligns with organisational objectives and stakeholder expectations.
- **User Requirements:** Insights into target audience needs, pain points, and expectations gathered directly from representative users.
- **Constraints:** Platform (iOS/Android), device support, regulatory compliance (GDPR, HIPAA), budget, and technology limitations.



5.2 The Requirements Gathering Process

Requirements gathering follows a structured sequence of steps to ensure that all stakeholder needs are captured completely and accurately.

Step 1 – Define the App's Idea and Purpose

Every requirements exercise begins by clearly articulating what problem the app solves and what value it delivers. Define the target audience, the core problem being addressed, and the app's primary goal. This anchors all subsequent decisions.

Step 2 – Identify and Engage Stakeholders

Identify everyone with a vested interest: end-users, business owners, product managers, developers, QA engineers, and legal/compliance stakeholders. For each stakeholder group, define their role in the requirements process and capture their key expectations early.

Step 3 – Run Market and Competitor Analysis

Conduct a SWOT analysis of competing products to identify gaps and opportunities. Rank competitors on key attributes (quality, pricing, features, UX, reputation) and determine where the new product can differentiate. This analysis shapes the product's unique value proposition and informs feature prioritisation.

Step 4 – Create User Personas and Scenarios

A user persona is a semi-fictional representation of the ideal user — defined by demographics, behavioural patterns, goals, frustrations, and motivations. Personas are then developed into scenario narratives or storyboards that describe how different users interact with the app to achieve their goals. These drive user story development.

Step 5 – Gather Functional and Non-Functional Requirements

With personas and scenarios established, systematically enumerate all functional requirements (what the app does) and non-functional requirements (how well it does it). Apply the MoSCoW prioritisation method:

- **Must Have:** Critical requirements — the app cannot launch without these.
- **Should Have:** Important but not launch-blocking.
- **Could Have:** Nice-to-have features that enhance UX if time and budget allow.
- **Won't Have:** Out of scope for the current release — deferred to future iterations.

5.3 Requirements Elicitation Techniques

Interviews

One-on-one or small group sessions with stakeholders. Semi-structured interviews combine guided questions with open exploration, uncovering deep user goals, mental models, and pain points that surveys miss.

Surveys and Questionnaires

Distributed to large audiences for quantitative data — feature priority rankings, satisfaction ratings, usage frequency. Tools: Google Forms, SurveyMonkey, Typeform. Best for validating assumptions at scale.

Contextual Inquiry / Observation

Watching real users perform tasks in their natural environment reveals unstated needs, workarounds, and workflow inefficiencies that users cannot or do not articulate in interviews.

Focus Groups

Facilitated group discussions with representative users. Generates a breadth of ideas quickly and identifies shared pain points and reactions to prototype concepts.

Workshops

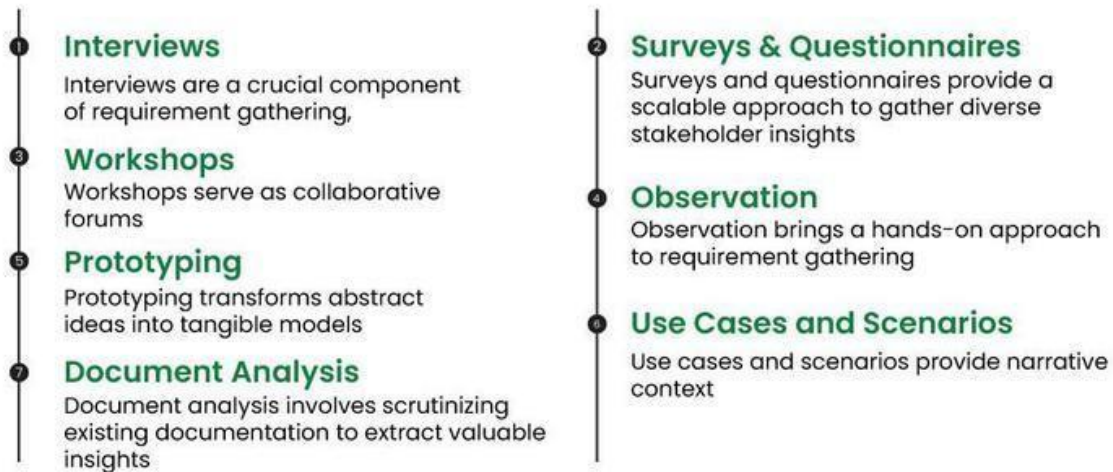
Structured collaborative sessions (e.g., Design Sprints) where stakeholders and developers define requirements together. Efficient for resolving conflicting viewpoints and aligning the team.

User Stories

Agile format: 'As a [user], I want to [action], so that [benefit].' User stories keep requirements user-centred, tied to business value, and small enough to be estimated and tested. Each story is accompanied by acceptance criteria that define 'done'.

Prototyping and Wireframing

Low-fidelity wireframes (sketches, clickable mockups) allow requirements to be visualised and validated before any code is written. High-fidelity prototypes enable usability testing. Prototyping is the single most effective technique for discovering implicit requirements and incorrect assumptions.



Requirement Gathering Techniques



5.4 Requirements Analysis Techniques

- **Gap Analysis:** Identifies the difference between the current state of a system and its target state.
- **Use Case Diagrams (UML):** Visualise actors and system interactions in a structured, notation-standard format.
- **Data Flow Diagrams:** Map how data moves through the system, highlighting potential gaps or bottlenecks.
- **Unified Modelling Language (UML):** Uses standardised diagrams (sequence, class, activity) to specify and communicate system design.
- **User Journey Maps:** Visualise the complete user experience from first contact to goal completion, identifying friction points.
- **Gantt Charts:** Outline the schedule of requirements analysis and development tasks.
- **MoSCoW Prioritisation:** Categorises requirements by importance to align development effort with business value.

5.5 Requirements Documentation

All gathered and analysed requirements must be formally documented in a Software Requirements Specification (SRS) document that includes:

- App idea statement and product overview
- Business requirements and objectives
- User personas and scenarios
- Detailed functional requirements with acceptance criteria
- Non-functional requirements (performance, security, scalability)
- UI/UX wireframes and prototypes

- Use case diagrams and user flow diagrams
- Cost-benefit analysis and MoSCoW-prioritised feature list
- Communication protocols and update procedures

5.6 Requirements Validation

Requirements must be validated before development begins:

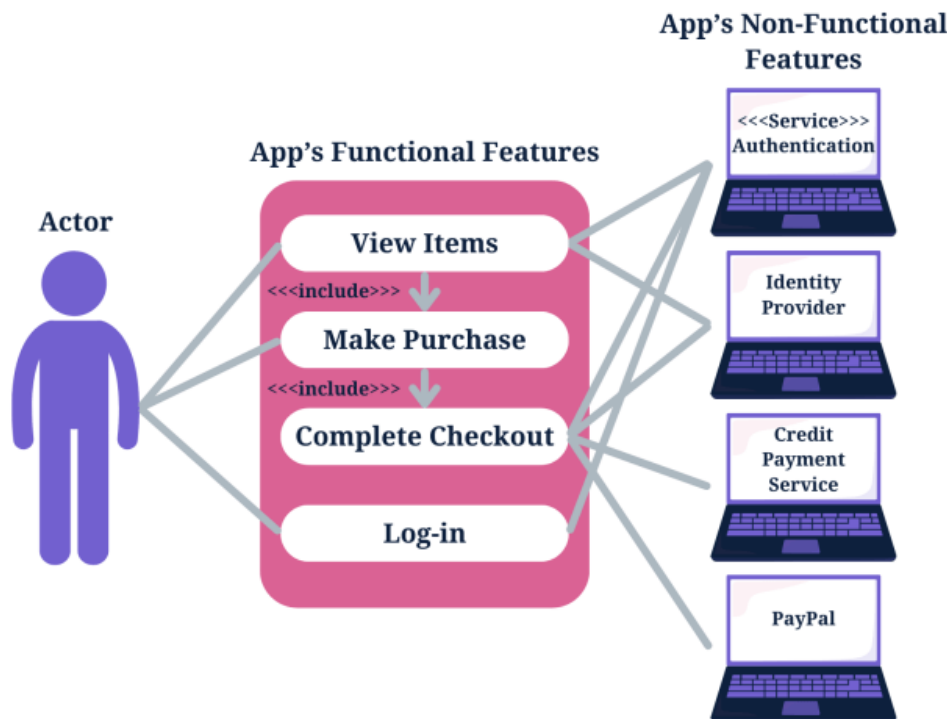
- Stakeholder reviews to verify completeness, correctness, and feasibility.
- Prototype/mockup testing with representative users to catch usability issues early.
- Traceability Matrix: maps each requirement to a design decision, a development task, and a test case.
- Iterative validation throughout development via Agile sprints — requirements evolve based on feedback.

CASE STUDY : How to Design a Use Case for App Requirements Documents

A use case diagram lets everyone at the development company visualize an overview of how users will interact with the app.

It's an overview that includes actors, how actors interact with the app, and the sequence of interactions actors will deploy.

Here's a simple example of a use-case diagram that shows an overview of how actors interact with functional features while non-functional features interact with the app from the back-end stack:



Write an App Requirements Document

Delivering a proper app requirements document means you need to know how to write a mobile app requirements document.

Mobile application development relies on the requirements document to design proper flow or the best app features and hit the right target audience.

a. Formulate the App's Idea Statement

Every app requirements document should include an idea statement that lets every stakeholder and software developer understand the document before diving into the details.

Start your app requirements document with a simple single-sentence statement that aligns with the app's idea.

b. Document All Relevant App Details

A detailed description of development plans in the requirements document is instrumental to completing the documentation.

A successful mobile app requirements document includes more than the details of an application and its functions for the development team.

Descriptions to Include in a Mobile Requirements Document

A mobile application requirements document should include a detailed description of functional and technical requirements and the app's functionality to properly capture and represent the project's scope.

- Business requirements
- User requirements
- Software requirements specification
- Technical specifications
- Functional specifications
- Non-functional requirements
- Hardware interfaces
- A list of must-have features
- Unique app features
- Internal and external interfaces
- Non-functional key metrics
- User stories
- Acceptance criteria

c. Prepare a Navigation Sequence

The development team requires a simple navigation sequence they can follow during software development.

The mobile app requirements document outlines the sequence in which the software development process flows.

d. Add Requirements Formats for Visuals

Add your user stories from a user's perspective and the use case overviews you designed to the app requirements document to help stakeholders and the development team understand every aspect of the app requirements document.

Successful software development means knowing the intended users.

e. Add Cost Optimization Details

The development team, stakeholders, and the client will appreciate a cost-benefit analysis to ensure cost optimization throughout the software development process. Business analysts also insist on adding a cost-benefit analysis to an app requirements document to meet the business needs and have a greater chance of success against competing apps.

f. Add Communication Protocols

Add communication methods for a collaborative process. Collaboration relies on dependable communication. The importance of effective communication in requirements gathering outlines why you need it.

g. Deploy Prototyping and Wireframing

Prototyping and wireframing let you design the user flow of user interfaces and basic app functions.

It also lets you test and validate layouts and transitions between app pages.

Here are the steps to wireframe your requirements for an app, which you will then validate in the next step:

1. Map the target user flow.
2. Sketch the flow's core part.
3. Set a mobile wireframe.
4. Determine the layout with boxes.
5. Use design patterns.
6. Add intended copy.

7. Connect the app's pages to design a flow.
8. Design a prototype.
9. Release the initial design to gather feedback.

h. Validate the App Requirements

Validation is a quality control process you use before launching the final product based on your requirements.

The prototype app collects feedback from stakeholders, and you can invite stakeholders to verify that the app meets the documented requirements. Use the feedback for the final step.

i. Apply Agile Methodology

Agile methodology in requirements-gathering means you'll always adapt the requirements document as per the feedback from stakeholders, testing, and initial product releases.

Agile methodology focuses on user experience and constant testing and validation to further improve your application.

6. Estimating Mobile App Development Cost

Accurately estimating the cost of a mobile application is critical for project planning, stakeholder alignment, and risk management. Poor estimation leads to budget overruns, scope creep, and unmet expectations. This section covers the key cost factors, estimation techniques, a practical step-by-step process, and typical cost benchmarks.

6.1 Key Cost Factors

App Complexity and Number of Features

Complexity is the single largest driver of cost. A simple app with 3–5 static screens and no backend costs far less than a complex app with real-time features, AI/ML capabilities, payments, and 25+ screens. The number, depth, and interdependency of features directly determines the development hours required.

Platform Choice

Building for iOS only or Android only is less expensive than building for both. Native development requires two separate codebases; cross-platform frameworks (Flutter, React Native) allow a single codebase for both platforms, reducing cost and time by 30–50%. The choice should be driven by target audience demographics and required performance.

UI/UX Design

Custom animations, branded design systems, accessibility compliance (WCAG), and multi-language support all increase design costs. A simple, standard UI may require 50–100 hours; an immersive custom design can exceed 200+ hours of design work.

Backend Development and API Integrations

Apps requiring server-side logic, cloud storage, relational databases, and third-party API integrations (payments, maps, social login, push notifications) carry significant additional cost. Backend infrastructure must also be designed for scalability and security.

Team Location and Composition

Hourly development rates vary widely by region:

- North America: \$100–200/hr
- Western Europe: \$70–150/hr
- Eastern Europe: \$40–80/hr
- Asia (India, Vietnam): \$20–60/hr



A full team includes project manager, UX designer, front-end developer(s), back-end developer(s), QA engineer, and DevOps — each role adding cost.

Maintenance and Post-Launch Costs

Mobile apps require ongoing investment after launch: bug fixes, OS update compatibility, server maintenance, feature additions, and app store fee management. Budget 15–20% of the initial development cost per year for maintenance.

6.2 Cost Estimation Techniques

App Development Cost Formula

The simplest baseline estimate:

$$\text{App Development Cost} = \text{Total Development Hours} \times \text{Hourly Rate}$$

Total hours encompasses all phases: planning, design, development, QA, and deployment. This formula is most accurate when the scope is well-defined.

Story Point Estimation (Agile)

Development tasks are decomposed into user stories and assigned relative complexity points (Fibonacci sequence: 1, 2, 3, 5, 8, 13...). The team's measured velocity (story points completed per sprint) allows projection of total timeline and cost. This method adapts well to evolving requirements and is the most widely used in Agile teams.

Three-Point Estimation (PERT)

Accounts for uncertainty by combining three scenarios:

$$\text{Expected} = (\text{Optimistic} + 4 \times \text{Most Likely} + \text{Pessimistic}) / 6$$

This produces a realistic weighted average and a confidence range rather than a single figure. Suitable for projects with significant unknowns.

Function Point Analysis (FPA)

Counts the functional components of the system — inputs, outputs, queries, data files, and external interfaces — then converts them to development hours using a productivity factor. Best for large, well-specified enterprise projects.

Analogous Estimation

Compares the new project against similar completed projects to derive a baseline estimate. Fast and practical, though less accurate without detailed scope definition. Useful for quick initial budgeting in client proposals.

6.3 Step-by-Step Cost Estimation Process

1. Define scope clearly – document all features, screens, user flows, and integrations.
2. Decompose into tasks – create a Work Breakdown Structure (WBS) with time estimates per task.
3. Identify team composition and apply hourly rates for each role.
4. Apply estimation technique – Story Points, PERT, or FPA as appropriate for the project type.
5. Add contingency buffer – 15–25% for unknowns, integration risks, and scope changes.
6. Include ongoing costs – hosting, maintenance (15–20%/year), app store fees (\$99/yr Apple, \$25/upload Google).
7. Review and adjust with stakeholders based on budget constraints and priority trade-offs.

6.4 Typical Cost Ranges

App Type	Complexity	Estimated Cost (USD)	Typical Timeline	Example
Simple	Basic UI, 3–5 screens, no backend	\$5,000 – \$30,000	1–3 months	To-do list, calculator
Medium	Login, DB, APIs, 10–20 screens	\$30,000 – \$100,000	3–6 months	Fitness tracker, food delivery

App Type	Complexity	Estimated Cost (USD)	Typical Timeline	Example
Complex	Real-time, AI, payments, 25+ screens	\$100,000 – \$300,000+	6–12 months	E-commerce, social media
Enterprise	Custom backend, high security, integrations	\$500,000+	12–24+ months	Banking, ERP systems

6.5 Cost Distribution Across Development Phases

- Planning & Requirements: ~10% of total budget
- Design & Prototyping: ~15%
- Development & Testing: ~50%
- Deployment & Launch: ~10%
- Post-Launch Maintenance (Year 1): ~15–20% of initial cost

6.6 Tactics for Reducing Development Costs

- MVP-first approach: Build only Must-Have features for the initial release, gather user feedback, then iterate.
- Cross-platform frameworks: Flutter or React Native can serve both iOS and Android from one codebase, cutting cost by up to 50%.
- Outsourcing strategically: Use nearshore or offshore teams for development while keeping product management and QA onshore.
- Clear requirements upfront: Every revision cycle due to unclear requirements adds cost. Thorough RE (Section 5) is the most cost-effective investment.
- Reuse existing components: Leverage open-source libraries, existing APIs (Firebase, AWS Amplify), and UI component kits.

7. Conclusion

Mobile app development is a rapidly evolving field that involves creating native, web, and hybrid applications tailored for different platforms. The choice of development approach depends on factors like performance, cost, and accessibility.

Cross-platform frameworks such as React Native, Flutter, Xamarin, and Ionic streamline development, enabling faster deployment and broader reach. Effective mobile app architecture, including MVC, MVVM, and Clean Architecture, ensures scalability, maintainability, and performance.

Cost estimation is crucial for budgeting, as factors like app complexity, platform choice, and development frameworks influence expenses. By selecting the right technologies and design patterns, businesses can optimize costs while delivering high-quality applications.

As mobile technology advances, developers must stay adaptable, leveraging modern frameworks and best practices to build efficient, scalable, and user-friendly applications.

References

- Types Of Mobile Applications:
<https://www.businessofapps.com/app-developers/research/types-of-mobile-apps/>
<https://www.geeksforgeeks.org/types-of-mobile-application-appium/>
<https://medium.com/@olivereric075/what-are-the-types-of-mobile-apps-d625b01e7fb0>
- Mobile Application Frameworks:
<https://www.geeksforgeeks.org/top-mobile-app-development-frameworks/>
<https://technostacks.com/blog/mobile-app-development-frameworks/>
- Estimating Mobile App Development Costs
[Estimating Mobile App Development Costs: A Comprehensive Guide - itCraft blog](#)
[How to Estimate Mobile App Development Cost \[Free Calculator Template\]](#)
[A Guide to Mobile App Development Cost Estimates](#)
[How to Calculate the Cost of Developing a Mobile App](#)
- Flutter. (2024). Flutter Documentation. <https://docs.flutter.dev/>
- React Native. (2024). React Native Documentation. <https://reactnative.dev/docs/getting-started>
- Ionic. (2024). Ionic Framework Documentation. <https://ionicframework.com/docs>
- Microsoft. (2024). .NET MAUI Documentation. <https://learn.microsoft.com/en-us/dotnet/maui/>
- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- itCraft Blog. (2024). Estimating Mobile App Development Costs: A Comprehensive Guide. <https://itcraftapps.com/>
- Clutch. (2024). Mobile App Development Cost Estimation Guide. <https://clutch.co/>
- Apple Developer. (2024). Swift Programming Language. <https://swift.org/documentation/>
- Google Developers. (2024). Kotlin for Android. <https://developer.android.com/kotlin/>